

Dokumentacja Techniczna Aplikacji FinanceTracker

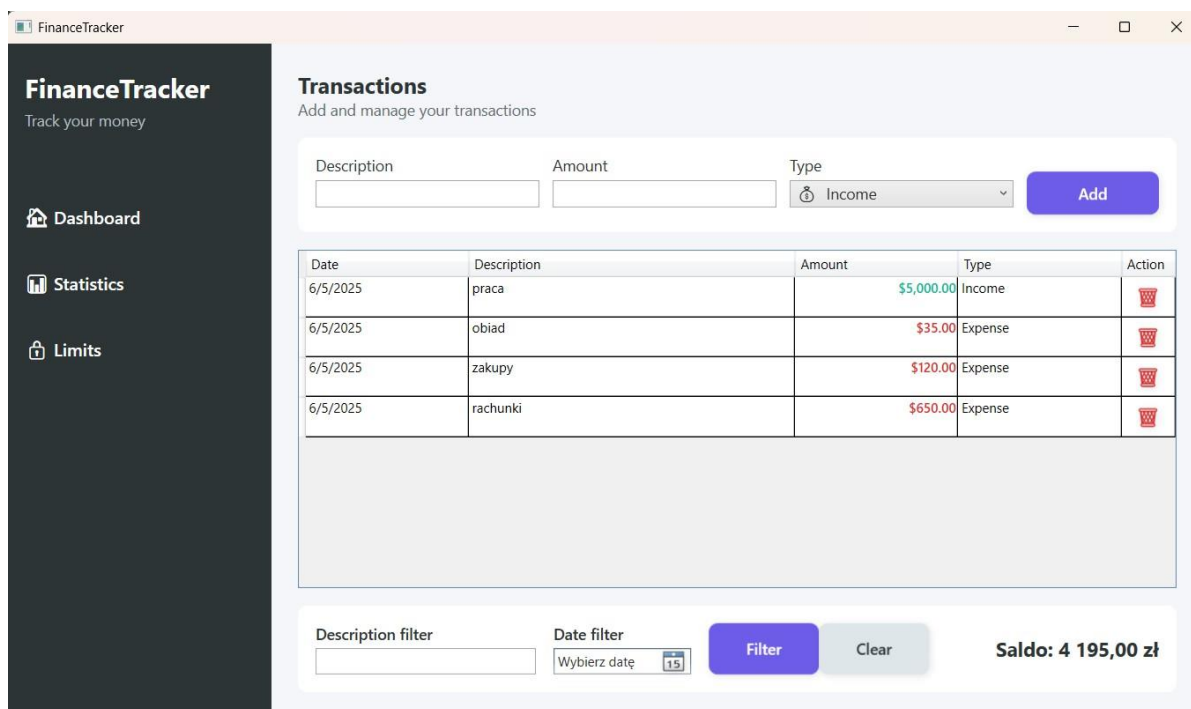
1. Opis ogólny

Aplikacja FinanceTracker to narzędzie desktopowe napisane w technologii WPF (.NET) do zarządzania finansami osobistymi. Pozwala na dodawanie i usuwanie transakcji, ustalanie limitów wydatków, analizę przychodów i kosztów oraz filtrowanie danych wg daty i opisu.

2. Widoki

Aplikacja posiada 3 główne sekcje:

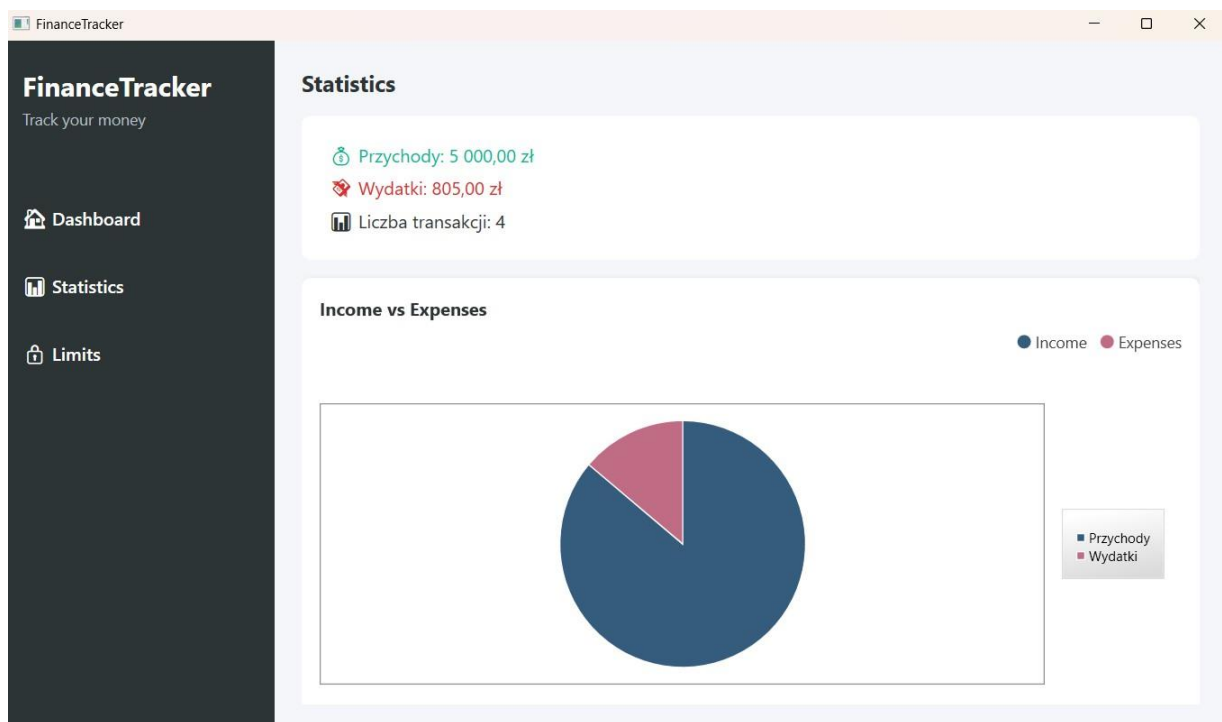
- **Budżet:** Dodawanie i lista transakcji.



The screenshot displays the FinanceTracker application window. On the left is a dark sidebar with the title "FinanceTracker" and the tagline "Track your money". Below the title are three menu items: "Dashboard" (with a house icon), "Statistics" (with a bar chart icon), and "Limits" (with a lock icon). The main area is titled "Transactions" with the subtitle "Add and manage your transactions". At the top of this area is a form to add a new transaction, consisting of input fields for "Description" and "Amount", a dropdown menu for "Type" (currently set to "Income"), and a blue "Add" button. Below the form is a table listing transactions. The table has five columns: "Date", "Description", "Amount", "Type", and "Action". It contains four rows of data, all dated "6/5/2025". The first row is for "praca" (work) with an income of "\$5,000.00". The next three rows are for "obiad" (lunch), "zakupy" (shopping), and "rachunki" (bills), all with expenses of "\$35.00", "\$120.00", and "\$650.00" respectively. Each row has a trash can icon in the "Action" column. At the bottom of the main area, there are filters: a "Description filter" input field, a "Date filter" with a calendar icon and the text "Wybierz datę", and buttons for "Filter" and "Clear". To the right of these filters, the current balance is displayed as "Saldo: 4 195,00 zł".

Date	Description	Amount	Type	Action
6/5/2025	praca	\$5,000.00	Income	
6/5/2025	obiad	\$35.00	Expense	
6/5/2025	zakupy	\$120.00	Expense	
6/5/2025	rachunki	\$650.00	Expense	

- **Statystyki:** Wykres i podsumowania.



- **Limity:** Ustawianie miesięcznych limitów.

The screenshot shows the 'Spending Limits' page of the FinanceTracker application. The left sidebar is identical to the previous screenshot. The main content area is titled 'Spending Limits' and contains a form to 'Set monthly spending limit'. The form has two input fields: 'Amount' with the value '5000' and 'Month' with the value '06.06.2025'. A 'Set Limit' button is positioned to the right of the 'Month' field. A small calendar icon is visible next to the date input.

Field	Value
Amount	5000
Month	06.06.2025

3. Klasy i Metody

3.1 Transaction

Reprezentuje pojedynczą transakcję finansową.

Własności:

- int Id – klucz główny.
- DateTime Date – data wykonania transakcji.
- string Description – opis transakcji.
- decimal Amount – kwota.
- TransactionType Type – typ: Income lub Expense.

Enum TransactionType:

- Income – przychód
- Expense – wydatek

3.2 SpendingLimit

Reprezentuje miesięczny limit wydatków.

Własności:

- int Id – klucz główny.
- decimal LimitAmount – maksymalna dozwolona kwota.
- DateTime Month – data reprezentująca miesiąc (zawsze pierwszy dzień miesiąca).

3.3 FinanceContext

Konfiguracja bazy danych SQLite poprzez Entity Framework.

DbSety:

- DbSet<Transaction> Transactions
- DbSet<SpendingLimit> SpendingLimits

Metoda:

- OnConfiguring(DbContextOptionsBuilder optionsBuilder) – ustawia źródło danych na finances.db.

3.4 TransactionManager

Klasa zarządzająca logiką aplikacji i komunikacją z bazą danych.

Konstruktor:

- `TransactionManager(FinanceContext? context = null)` – pozwala na przekazanie zewnętrznego kontekstu lub utworzenie nowego.

Metody:

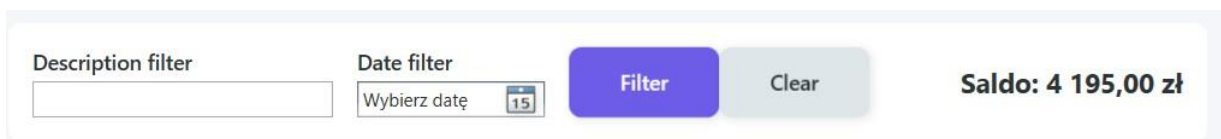
- `AddTransaction(Transaction transaction)`
 - Dodaje transakcję do bazy danych.
- `DeleteTransaction(Transaction transaction)`
 - Usuwa transakcję na podstawie Id.
- `GetAllTransactions()`
 - Zwraca listę wszystkich transakcji z bazy.
- `GetBalance()`
 - Oblicza saldo jako suma przychodów minus suma wydatków.
- `InitializeDatabase()`
 - Tworzy bazę danych jeśli nie istnieje.
- `GetLimitForCurrentMonth()`
 - Zwraca limit wydatków ustawiony na bieżący miesiąc.
- `GetTotalExpensesThisMonth()`
 - Sumuje wydatki z bieżącego miesiąca.
- `CanAddExpense(decimal amount)`
 - Sprawdza, czy dodanie nowego wydatku nie przekroczy ustalonego limitu.

3.5 MainWindow.xaml.cs

Plik odpowiedzialny za logikę interfejsu użytkownika.

Główne elementy i metody:

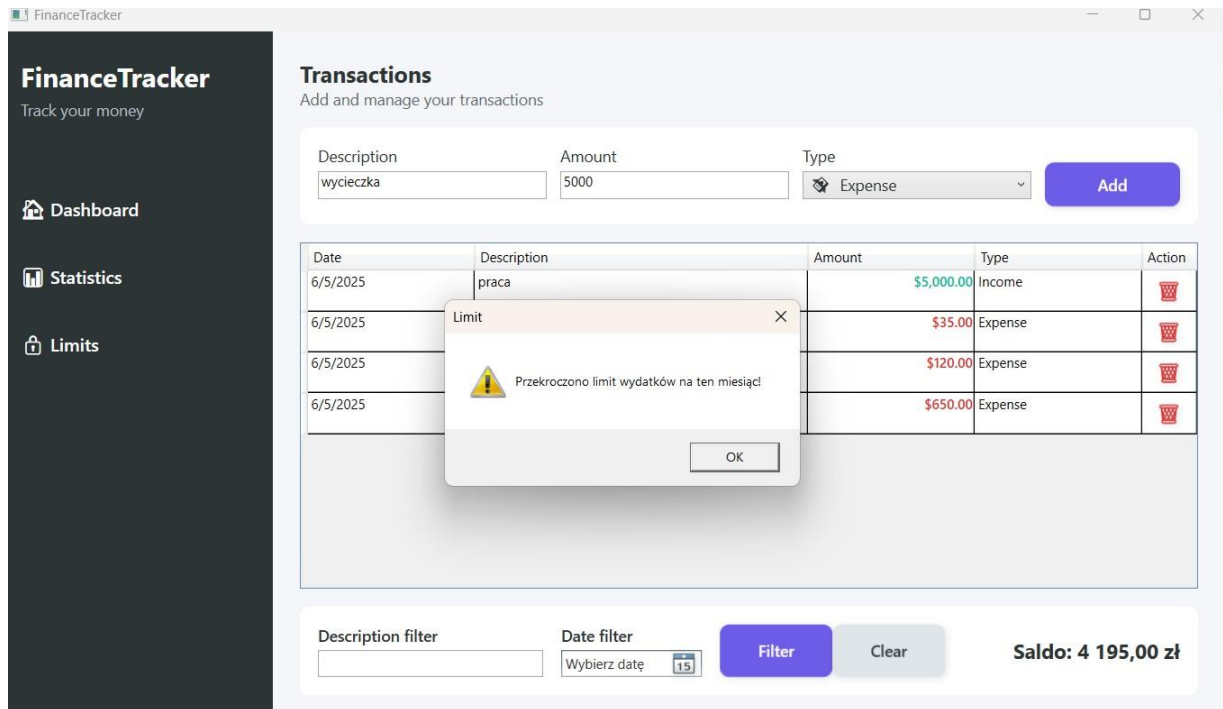
- MainWindow()
 - Inicjalizacja komponentów, bazy danych, załadowanie danych do DataGrid.
- AddTransaction_Click(object sender, RoutedEventArgs e)
 - Obsługuje kliknięcie przycisku dodawania transakcji.
 - Waliduje dane wejściowe, sprawdza limit i zapisuje transakcję.
- DeleteTransaction_Click(object sender, RoutedEventArgs e)
 - Usuwa zaznaczoną transakcję z widoku i bazy danych.
- UpdateBalance()
 - Uaktualnia wartość salda w interfejsie.
- UpdateStatistics()
 - Oblicza i wyświetla statystyki (dochody, wydatki, liczba transakcji) oraz generuje dane do wykresu.
- ShowBudgetView(), ShowStatsView(), ShowLimitsView()
 - Przełączanie widoczności sekcji interfejsu.
- FilterTransactions_Click()
 - Filtrowanie transakcji wg opisu i daty.



The screenshot shows a user interface for filtering transactions. It features a 'Description filter' input field, a 'Date filter' section with a 'Wybierz datę' button and a calendar icon displaying the date '15', a blue 'Filter' button, a grey 'Clear' button, and a 'Saldo: 4 195,00 zł' label.

- ClearFilters_Click()
 - Resetuje filtry i przywraca wszystkie dane.

- SetLimit_Click()
 - Zapisuje lub modyfikuje limit wydatków dla wybranego miesiąca.
 - Jeśli limit zostanie przekroczony jest wyświetlany odpowiedni komunikat



3.6 UnitTest1.cs

Zawiera testy jednostkowe korzystające z InMemory EF Core.

Testy:

- AddTransaction() – testuje poprawność dodawania transakcji.
- GetBalance() – testuje poprawne obliczenie salda.
- LimitExpenses() – sprawdza działanie walidacji limitu wydatków.

4. Baza danych

Używa SQLite jako lokalnego magazynu danych.

- Automatyczna inicjalizacja przez EnsureCreated().
- Tabele: Transactions, SpendingLimits
- Każda transakcja posiada unikalne Id.

5. Statystyki i wykres

Użytkownik może:

- Przejrzeć sumę przychodów i wydatków.
- Zobaczyć liczbę transakcji.
- Obejrzeć wykres kołowy (XAML PieSeries).

6. Testowanie

- Testy jednostkowe są odseparowane od fizycznej bazy danych.
- Używają UseInMemoryDatabase() dla szybkości i niezależności.

7. Główne pakiety NuGet

^ Pakiety najwyższego poziomu (2)		
	Microsoft.EntityFrameworkCore.Sqlite przez: Microsoft SQLite database provider for Entity Framework Core.	9.0.4 9.0.5
	System.Windows.Controls.DataVisualization.Toolkit przez: Microsoft Corporation WPF Toolkit Data Visualization Controls	4.0.0

^ Pakiety najwyższego poziomu (4)		
	Microsoft.EntityFrameworkCore.InMemory przez: Microsoft In-memory database provider for Entity Framework Core (to be used for testing purposes).	8.0.0 9.0.5
	Microsoft.NET.Test.Sdk przez: Microsoft The MSbuild targets and properties for building .NET test projects.	17.9.0 17.14.1
	MSTest.TestAdapter przez: Microsoft The adapter to discover and execute MSTest Framework based tests.	3.0.2 3.9.1
	MSTest.TestFramework przez: Microsoft This is MSTest V2, the evolution of Microsoft's Test Framework.	3.0.2 3.9.1

8. Autor

- Paweł Kulesza
- Dominik Pakuła